



Score breakers

SYNOPSIS LAVET AF:

EMIL TOFTBO VAN DIJK

LINK TIL GITHUB: [HTTPS://GITHUB.COM/B6RNARD/SCORE_BREAKERS](https://github.com/B6RNARD/SCORE_BREAKERS)

Skole: Slotshaven gymnasium

Fag: Programmering B

Lærer: Mikkel Kirkeby Mosthaf

Dato for aflevering: 27/03/26

Applikation af: Emil Toftbo Van
Dijk og Cornelius Struer
Lindemann

Indhold

Eksamensopgave og krav:	2
Projektbeskrivelse	3
Kravspecifikation.....	3
AI:.....	5
Funktionsbeskrivelse:	5
Spillende	6
Leaderboards.....	9
Flask server	10
Login/Brugere	11
Dokumentation:	12
Kode:	14
Scores:	14
Login:	15
T-rex run:.....	16
Tests:	17
Konklusion:	17
Bilag:.....	19

Eksamensopgave og krav:

Tema 1. Website med eget tema

"I skal udvikle et program eller en applikation som demonstrerer brug struktureret brug af trelagsmodellen. Det vil sige at løsningen skal indeholde et data-, et logik- og et præsentationslag. Datagrundlaget og præsentationslaget er valgfrit - I bestemmer altså selv hvad løsningen handler om og hvordan den præsenteres for brugeren."

Overordnet Krav:

Non-funktionelle krav:

- Data: der skal indgå en datakilde (struktureret datafil, eksternt API, database eller lign)
- Logik: det midterste lag skal "oversætte" mellem de to andre lag
- der skal ske en eller anden transformation af data.
- OOP: der skal indgå objektorienteret teknikker og anvendes mindst en selvskrevet klasse
- AI: til AI-genereret kode benyttes instruktionsfiler som afspejler projektbeskrivelsens målsætninger og krav
- Kodebase: Kode skal lagres i selvstændigt repo i offentligt tilgængeligt versioneringssystem
- Kodebase: Kode skal være kommenteret, velstruktureret og alle irrelevante filer skal være fjernet

Funktionelle krav :

- Løsningen skal være interaktiv
- brugeren skal have nogle muligheder
- Præsentation: applikation skal præsentere data for brugeren på struktureret vis
- Brugerinput skal valideres

Projektbeskrivelse

Vores projekt tager udgangspunkt i tema 1 og går ud på at udvikle en applikation med en arkadeinspireret brugerflade, hvor brugeren kan browse og scrolle gennem et udvalg af klassiske spil. Applikationen skal have et visuelt udtryk, der minder om en traditionel arkadehal med farver, simple menuer og tydelige spilvalg.

Brugeren skal kunne oprette en profil og logge ind. Når man er logget ind, får man adgang til cirka tre klassiske spil, for eksempel spil som Pac-Man, Tetris og Space Invaders. Spillene udvikles som vores egne versioner for at copyright ikke er et problem, med grundlæggende funktionalitet og arkadeinspireret design. Hjemmesiden skal gøre det muligt for brugeren at gemme highscores, så man kan følge sin score og evt. sammenligne highscores med andre brugere.

Kravspecifikation

Her opsætter vi vores specifikke krav til vores projekt og hvordan vi vurderer deres vigtighed for projektet.

Brugeren skal kunne oprette en konto og logge ind:

Vi ønsker at implementere en profilside, hvor brugeren både kan oprette nye profiler og vælge mellem eksisterende. Derudover vil vi vise profilnavnet sammen med highscores, så det tydeligt fremgår, hvem der har opnået hvilke resultater.

Brugeren skal kunne vælge mellem minimum tre spil:

Da applikationen fungerer som en arkade hub eller liggende er det vigtigt med flere spil for at skabe variation og en bedre brugeroplevelse. Ideen er inspireret af platforme som Y8, Friv og Steam, hvor brugeren har adgang til flere spil et sted.

Spillene skal have en fungerende spilmekanik (bevægelse, point, game over):

Spillene skal være fuldt spilbare og ikke blot prototyper. De skal indeholde grundlæggende funktioner som bevægelse, point og game over, så brugeren får en reel spiloplevelse og mulighed for at opnå highscores.

Systemet skal registrere og gemme brugerens highscore:

Dette er et centralt krav, da hele idéen med applikationen er at kunne konkurrere med andre. Derfor er det vigtigt, at highscores automatisk bliver gemt og knyttet til den enkelte bruger.

Brugeren skal kunne se sin egen highscore på sin profil:

Dette er et mindre kritisk krav, da man også kan se sine scores på leaderboardet. Det bidrager dog til en bedre brugeroplevelse ved at samle information ét sted og give en mere komplet profil.

Gruppefordelingen:

Jeg fokuserer primært på Logik laget og vil derfor sørge for at der er en effektiv og forhåbentlig nem oversættelse fra data laget til præsentationslaget.

Cornelius fokuserer primært på GUI og vil have fokus på at alle interaktionselementer virker effektive og ser behageligt ud og følger temaet.

Krav:

Krav	Vurdering
Brugeren skal kunne oprette en konto og logge ind	Need-to-have
Brugeren skal kunne vælge mellem minimum tre spil	Nice-to-have
Spillene skal have en fungerende spilmekanik (bevægelse, point, game over)	Need-to-have
Systemet skal registrere og gemme brugerens highscore	Need-to-have
Brugeren skal kunne se sin egen highscore på sin profil	Nice-to-have

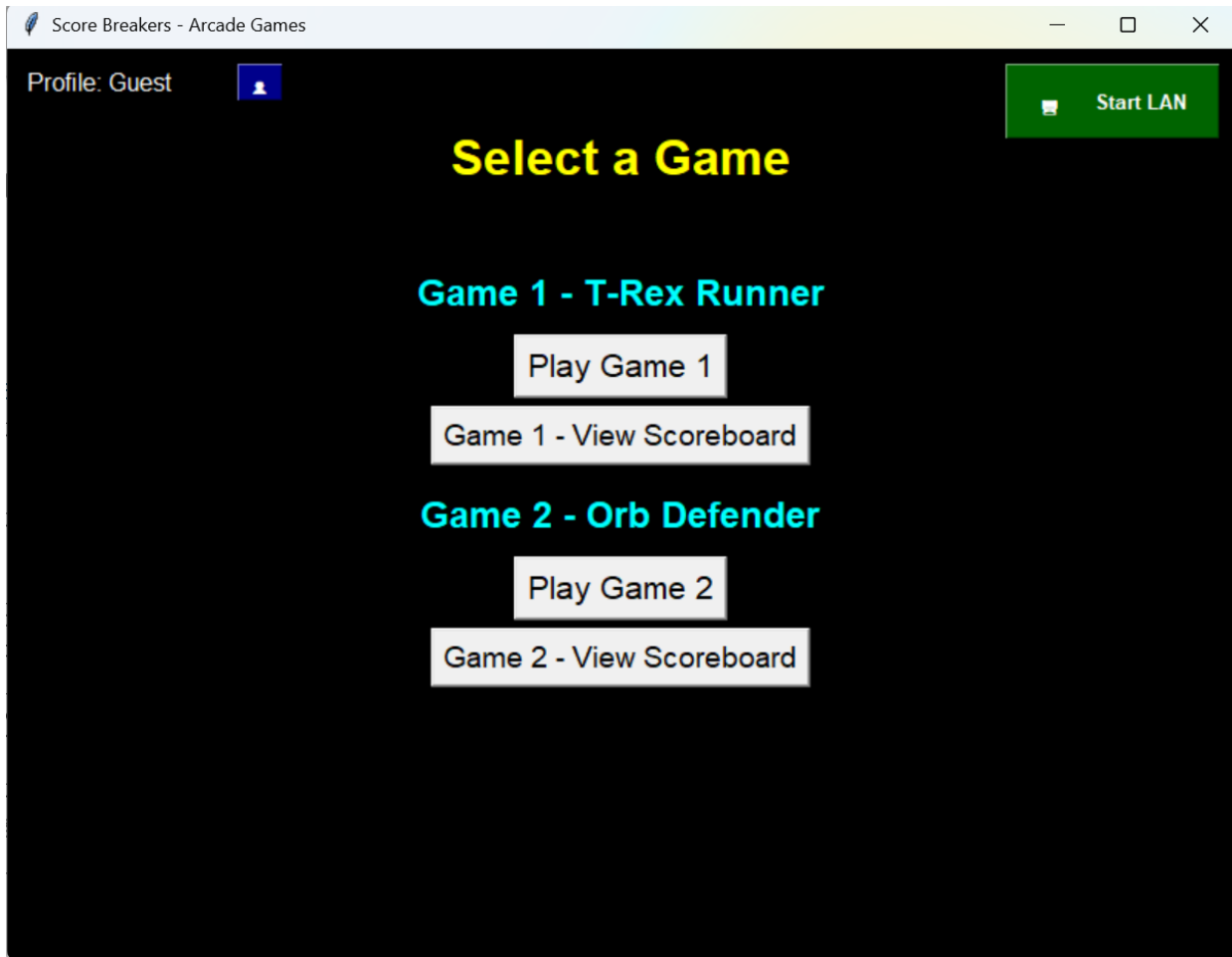
AI:

Vi har primært brugt AI til produktionen af selve spillene da det ikke er vores fokus, som ligger i vores scoreboard- og login system. Andet vi har benyttet AI til er noget af styling og design, men det ville primært være til en grov opsætning som der bliver bygget ud fra. Vi vil gøre dette ved at anvende en instruktionsfil, som er skrevet i workspacet hvilket er et sæt instruktioner som AI'en følger som et set retningslinjer vi har fastlagt. Det vil blandt andet omfatte dokumentation af AI-brug, hvor både prompts, svar og datoer for vores brug af AI bliver registreret.

Funktionsbeskrivelse:

Vores program/applikation består af mange forskellige dele men kan opdeles i disse dele, selve spillene, scores og scoreboard, vores flask server og login/brugerne. Vi har benyttet en readme som brugerne kan benytte som en instruks på hvordan de skal gøre hvis de ville se hinandens scores.

Spillende



Figur 1 menu

Når man køre koden i main.py starter applikationen hvor man ser en form for hovedmenu hvor man ser begge spilbare spil, ens bruger og mulighed for at lave en oppe i venstre hjørne. I højre side ser man hvor man kan starte LAN hvilket er hvor man har muligheden for at dele sine scores med andre på samme net.

Der er i øjeblikket to forskellige spil tilgængelige som man kan spille og opnå highscores i. Disse scores bliver automatisk gemt og vist i Scoreboardet. For at starte et spil skal man klikke på knappen “Play Game” under det ønskede spil. Det første spil vi har lavet er T-rex runner hvilket er inspireret af det originale T-rex runner som kommer op når man ikke har noget internet på Google.



Figur 2 T-rex runner

I dette spil skal man hoppe over forhindringer hvor scoren er baseret på, hvor længe man overlever, fremfor hvor mange forhindringer man passerer. Hastigheden ændrer sig ikke undervejs, så udfordringen består udelukkende i at holde sig i live så lang tid som muligt.

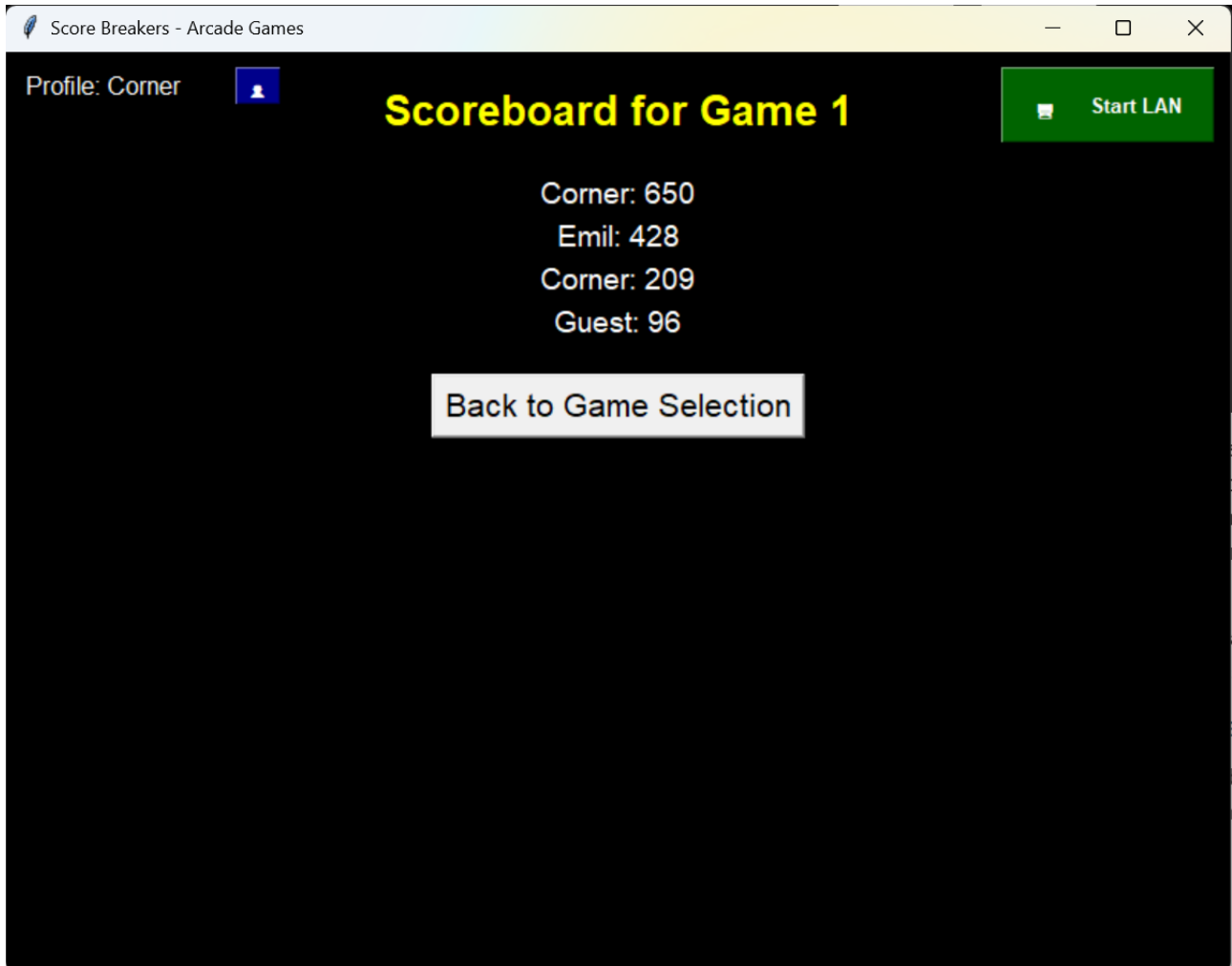


Figur 3 Orb defender

Det andet spil som vi lavede er et Space Invaders inspireret spil hvor man er den lyseblå platform/figur og skal skyde de forskelligt farvet kugler/bomber der falder fra himlen, målet er at skyde bomberne inden de rammer jorden hvilket gør du taber spilet. Ens score er udregnet på hvor mange bomber man får skudt inden der er en der når at ramme jorden.

Leaderboards

Vores scoreboards er en mulighed ligesom spillene at se fra hovedmenuen, det har den simple funktion at tage de gemte scores og vise dem på scoreboard skærmen



Figur 4 Scoreboard

Vi har igen valgt et meget simpelt design til scoreboardet for at bevare en GUI, der minder om en klassisk arkade. På scoreboard skærmen bliver der vist scores i rækkefølge efter højeste score først sammen med navnet på spilleren så man nemt kan finde sine egne og andres resultater hvis man spiller på samme computer.

Flask server

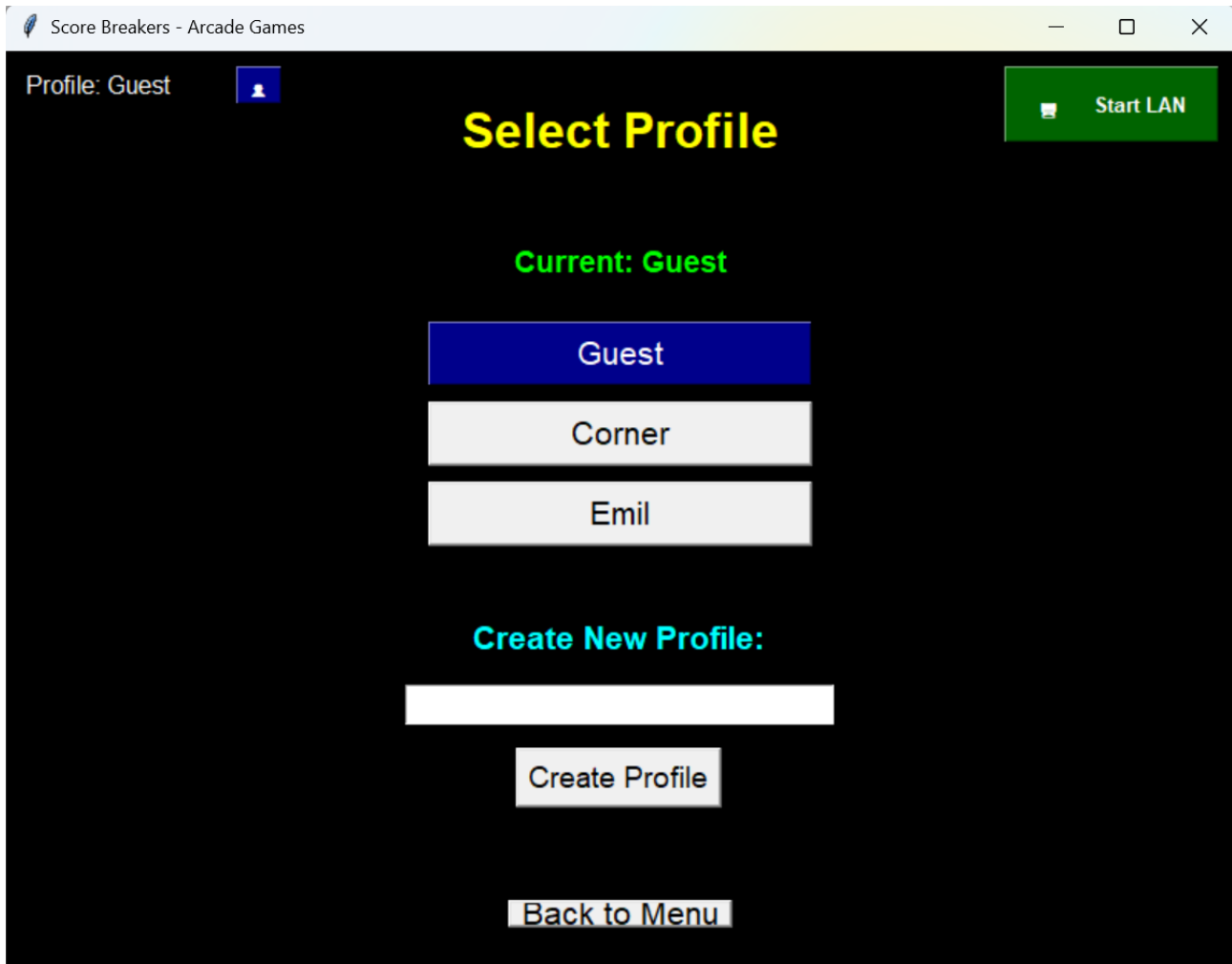
Hvis man spiller på hver sin computer så har vi også gjort det muligt at dele sine score via vores LAN knap der også kan ses på hovedmenuen.



Figur 5 Flask server

Når man trykker på knappen starter LAN serveren som starter en lokal hjemmeside der kører med falsk som alle der kører på det samme wifi kan se så længe de har URL'en til siden der displayer spillernes score, hjemmesidens URL til flask serveren bliver og vist oppe i højre hjørne af applikationen når man trykker på den grønne start LAN knap.

Login/Brugere



Figur 6 Login

Når man trykker på profilknappen åbner en side med muligheden for at forblive en gæst, gå ind på en tidligere oprettet bruger eller oprette en ny bruger med deres eget score tracking. Her kan man se, hvilken profil der er aktive. Siden det hele køre lokalt kræves der ikke adgangskode til profilerne, samt så er det ikke er muligt at slette hverken profiler eller rekorder.

Dokumentation:

I dette projekt er applikationen struktureret efter trelagsmodellen, hvilket er en opdeling op i et datalag, logiklag og præsentationslag (GUI). Den her opsætning hjælper med til en klar ansvarsfordeling og gør det både nemmer med at vedligeholde samt at teste dele af kode da de enkelte lag kan udvikles og evalueres uafhængigt af hinanden.

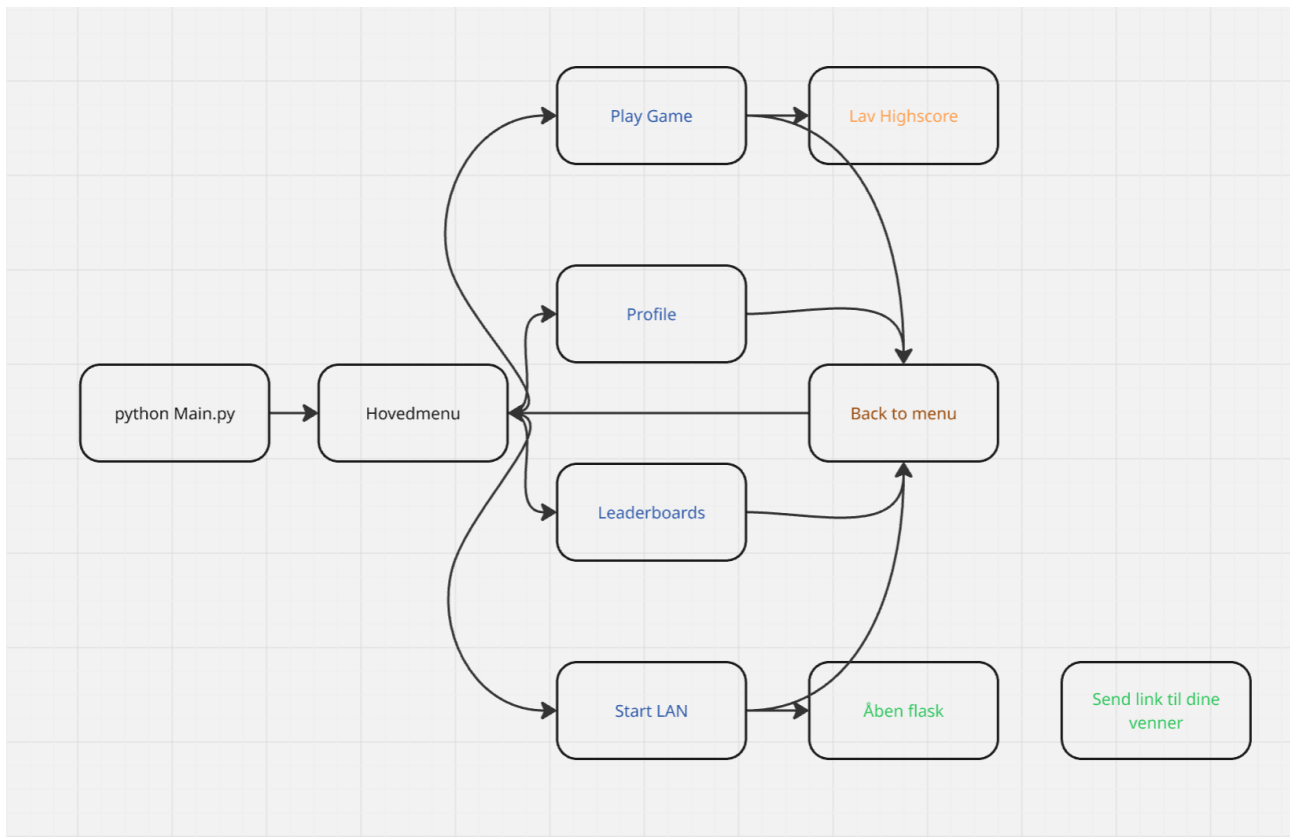
Datalaget har ansvar for at håndtere lagring og hentning af data. Her gemmer vi fx brugerprofiler og highscores i JSON-filer. Vi har valgt JSON, fordi det er let at arbejde med i Python og giver en overskuelig datastruktur. Funktionerne i datalaget sørger for at oprette, opdatere og hente data, så resten af programmet altid arbejder med korrekt information og regler.

Logiklaget fungerer som en bro eller oversættelse mellem data og brugergrænsefladen. Det er her selve spilmekanikken ligger så som pointberegning, styring af spillet og hvordan det virker samt håndtering af hvornår et spil lukkes. Logiklaget sørger også for at behandle data, inden det sendes videre til GUI-laget, så det vises korrekt for brugeren så som JSON filerne.

GUI-laget består af programmets brugergrænseflade, herunder menuer og spil, som er udviklet i Tkinter. Her har vi haft fokus på et simpelt og overskueligt design inspireret af klassiske arkadespil. GUI-laget registrerer brugerens input og sender det videre til logiklaget, som derefter håndterer det. Derudover har vi implementeret en Flask-server, som gør det muligt at dele highscores over et lokalt netværk. Hvilket udvider programmets funktionalitet og gør det muligt at se dataen fra flere enheder så længe der er på samme netværk.

Programmet bliver startet i main.py, som samler de forskellige dele og sørger for, at de arbejder sammen. Her initialiserer man applikationen, og forbindelsen mellem lagene etableres. Overordnet fungerer programmet ved, at brugeren interagerer med GUI-laget, hvorefter logiklaget behandler inputtet og enten opdaterer data eller returnerer et resultat, som vises i brugerfladen.

Der er også det Flowchart som giver et generelt overblik over hvordan vores program fungerer.



Figur 7 Flowchart

Kode:

Her kommer jeg til at gå igennem steder jeg har arbejdet med logik laget for enten at oversætte fra data til brugerflade eller omvendt.

Scores:

```
55 def add_score(self, game_key: str, player: str, score: int) -> None:
56     """
57     Add a score entry for a game.
58
59     - game_key: e.g. "game1", "game2"
60     - player: profile name
61     - score: numeric score
62     """
63     if game_key not in self._data:
64         self._data[game_key] = []
65
66     self._data[game_key].append({"player": player, "score": int(score)})
67
68     # Sort descending by score (highest first)
69     self._data[game_key].sort(key=lambda s: s.get("score", 0), reverse=True)
```

Figur 8 Kode udsnit (Scores)

Her ser vi for eksempel hvordan programmet opdaterer rekorder med oplysninger om brugernavn, hvilket spil der er tale om, samt selve scoren, som det fremgår af den viste funktion. Vi ser også hvordan der bliver tilføjet nye scores samt hvilken rækkefølge scores skal stå i. på linje 66 ser vi fx hvordan registreringen af en score ser ud med:

```
({"player": player, "score": int(score)})
```

Figur 9 Kode udsnit

Vi ser også hvordan rækkefølgen af listen er ordnet i linje 66 og 69

Login:

```
143 def create_new_profile(self) -> None:
144     """Create a new profile if name is valid."""
145     new_name = self.new_profile_entry.get().strip()
146
147     if not new_name:
148         return
149
150     if new_name in self.profiles:
151         # Profile already exists, just select it
152         self.select_profile(new_name)
153         return
154
155     # Add new profile
156     self.profiles.append(new_name)
157     save_profiles(self.profiles)
158
159     # Select the new profile
160     self.select_profile(new_name)
```

Figur 10 Kode udsnit (login)

her har vi et eksempel på hvor jeg har brugt logik laget til at lave nye brugere, vi ser bl.a. hvad der sker hvis man prøver at lave en bruger uden et navn i linje 147. vi ser også at hvis man skriver navnet på en tidligere bruger der er registreret laver den ikke en kopi men vælger i stedet den tidligere bruger. Vi ser også at når der bliver registreret en ny bruger/user gemmer den det i en JSON fil og vælger den.

T-rex run:

```
246     def _game_over(self) -> None:
247         """Stop the game and show 'Game Over' and store the score."""
248         self.is_running = False
249
250         # Save score via ScoresDB if available
251         if self.scores is not None:
252             try:
253                 # "game1" key is used by the scoreboard
254                 self.scores.add_score("game1", self.current_profile, self.score)
255                 self.scores.save()
256             except AttributeError:
257                 # If scores is not a ScoresDB, just ignore (template fallback)
258                 pass
259
260         self.game_over_text_id = self.canvas.create_text(
261             self.canvas_width // 2,
262             self.canvas_height // 2,
263             text=f"GAME OVER\nScore: {self.score}",
264             fill="red",
265             font=("Arial", 24, "bold"),
266         )
```

Figur 11 Kode udsnit (T-rex run)

Det her billede viser, hvordan spillet håndterer afslutningen af en runde. Når spillet slutter, stoppes spil-loopet, så spillet ikke længere opdateres. Derefter gemmes spillerens score, hvis en score-database er tilgængelig. Scoren knyttes til den aktuelle brugerprofil og det relevante spil og gemmes, så den senere kan vises i leaderboardet.

Til sidst viser den en "Game Over" besked med den opnåede score. Det er mere en visuel del, men hænger sammen med logikken, da den formidler resultatet af spillets afslutning.

Tests:

For at sikre, at vores program fungerer, som det skal, har vi testet de vigtigste funktioner. Vi har især haft fokus på logiklaget og datalaget, da det er her, de centrale funktioner ligger.

Vi har blandt andet testet, om highscores bliver gemt korrekt, om data kan hentes fra JSON-filer, om profiler opdateres rigtigt og om leaderboardet bliver sorteret korrekt. Vores tests er bygget op efter AAA-princippet (Arrange, Act, Assert), hvor vi først sætter nogle testdata op, derefter udfører en handling og til sidst tjekker, om resultatet er som forventet. Et eksempel er, at vi indsætter en ny highscore og derefter kontrollerer, om den bliver gemt rigtigt og vises korrekt på leaderboardet, både i falsk serveren og selve applikationen.

Det er sværere at teste GUI'en automatisk, fordi den afhænger af brugerens input. Derfor har vi også lavet manuel test, hvor vi selv har gennemgået programmet og prøvet alle funktioner for at sikre, at alt virker, som det skal.

Alt i alt viser vores test, at programmet fungerer stabilt uden fejl i de vigtigste funktioner. Vi har også fået feedback fra nogle af vores klassekammerater, som sagde, at vi rammer den klassiske arkade-stil godt, hvilket var det, vi gik efter.

Konklusion:

I dette projekt har vi udviklet en applikation i form af en game hub, hvor brugere kan spille forskellige spil, sætte highscores og konkurrere med hinanden.

Formålet med projektet var at lave et program, der demonstrerer brugen af trelagsmodellen, og det mener vi, at vi er lykkedes med. Opdelingen i datalag, logiklag og GUI-lag har gjort det nemmere at strukturere vores kode og skabe overblik over, hvordan de forskellige dele af programmet hænger sammen.

Vi har opfyldt de vigtigste krav, vi satte i starten af projektet, herunder at brugeren kan spille spil, få registreret sine highscores og se dem på et leaderboard. Derudover har vi implementeret profiler, så man kan skifte mellem brugere, samt en LAN-funktion ved hjælp af Flask, som gør det muligt at dele highscores over et lokalt netværk. Dette gør programmet mere interaktivt og giver mulighed for en form for konkurrence mellem brugere.

Arbejdsprocessen har givet os en bedre forståelse for, hvordan man opbygger større programmer, og hvordan de forskellige lag arbejder sammen. Især logiklaget har været centralt, da det styrer spilmekanikken, håndterer point og sørger for, at data bliver behandlet korrekt, før det gemmes eller vises.

Brugen af AI har været en hjælp til at forstå områder, vi ikke havde så meget erfaring med, som for eksempel Flask og JSON-håndtering. Vi har dog brugt det kritisk som et værktøj til inspiration og støtte fremfor at lade det lave arbejdet for os.

Der har været nogle begrænsninger i projektet, blandt andet i forhold til test af GUI, da det er svært at automatisere. Derfor har vi i stedet testet disse dele manuelt. De funktioner, vi har kunnet teste automatisk, har fungeret som forventet uden fejl.

Samlet set vurderer vi, at projektet er en succes. Programmet opfylder opgavekravene og viser en god forståelse for både programmering og programstruktur.

Bilag:

Figur 1 menu.....	6
Figur 2 T-rex runner	7
Figur 3 Orb defender	8
Figur 4 Scoreboard.....	9
Figur 5 Flask server	10
Figur 6 Login	11
Figur 7 Flowchart	13
Figur 8 Kode udsnit (Scores)	14
Figur 9 Kode udsnit	14
Figur 10 Kode udsnit (login)	15
Figur 11 Kode udsnit (T-rex run)	16